

ARCHITECTURE

- docker CLI → **Engine API** (/var/run/docker.sock, TCP+TLS, ssh://) → **dockerd** → **containerd** → **shim** → **runc** → kernel.
- Container = process + **namespaces** (pid, mnt, net, uts, ipc, user, cgroup) + **cgroups v2** + capabilities + seccomp + AppArmor/SELinux.
- **Socket access** = **root** on the host (so is the docker group). Rootless mode avoids it.
- **Oci**: image-spec (index → manifest → config + layers), runtime-spec (bundle for runc), distribution-spec (registry API).
- **BuildKit** builds (default since 23.0; docker build = buildx build).
- Engine 29: containerd image store default on fresh installs; cgroup v1 deprecated; Content Trust removed; nftables opt-in (firewall-backend).
- Desktop = Linux VM (Docker VMM, Apple VF, WSL 2, Hyper-V); host.docker.internal reaches the host.

IMAGES

```
[registry/][namespace/]repo[:tag]
[sha256:...] — default
docker.io/library/:latest. Tags
move, digests don't; never deploy
latest.

docker pull postgres:18.1
docker image ls          # tree
view; --all, --tree
docker image inspect IMG #
docker image history IMG #
layer per step
docker tag IMG reg/ns/app:1.4.0
docker push reg/ns/app:1.4.0
docker image save -o a.tar IMG
docker image load -i a.tar
docker buildx imagetools inspect
IMG
docker image prune -a --filter
until=168h

commit/export: debugging only — not
reproducible.
```

DOCKERFILE INSTRUCTIONS

Instruction	Use
#	latest 1.x frontend; also syntax=docker/dockerfile:1
FROM img AS name	new stage; --platform=\$BUILDPLATFORM
RUN	new layer; shell or exec form; here-docs
COPY / ADD	--from --chown --chmod --link --parents --exclude; ADD --checksum
WORKDIR / USER	cwd / non-root UID (USER 10001)
ENV / ARG	runtime env / build-time only; no secrets in either
EXPOSE	documentation only; -p publishes
HEALTHCHECK	--interval --start-period CMD ...
LABEL	org.opencontainers.image.*; MAINTAINER is deprecated
VOLUME	rarely needed
STOPSIGNAL	
SHELL	
ONBUILD	
ENTRYPOINT + CMD	Runs
["java","-jar","a.jar"] + ["-x"] (args replace CMD) ["-x"]	
shell form java -jar a.jar	/bin/sh -c ... — PID 1 is sh, SIGTERM lost, CMD ignored
RUN --mount	For
type=cache, Maven/npm caches target=/roo across builds t/.m2	
type=bind, s read context files, no ource=pom.x layer ml	
type=secret file in /run/secrets/s, ,id=s never stored	
type=ssh / agent forwarding / scratch type=tmpfs RAM	

MULTI-STAGE SKELETON (SPRING BOOT)

```
# syntax=docker/dockerfile:1
FROM eclipse-temurin:25-jdk AS build
WORKDIR /src
COPY mvnw pom.xml ./
COPY .mvn/ .mvn/
RUN --mount=type=cache,target=/root/.m2 \
    ./mvnw -B -ntp dependency:go-offline
COPY src/ src/
RUN --mount=type=cache,target=/root/.m2 \
    ./mvnw -B -ntp package -DskipTests \
    && cp target/*.jar /tmp/application.jar
RUN java -Djarmode=tools -jar /tmp/application.jar \
    extract --layers --destination /x

FROM eclipse-temurin:25-jre
WORKDIR /application
COPY --from=build /x/dependencies/ ./
COPY --from=build /x/spring-boot-loader/ ./
COPY --from=build /x/snapshot-dependencies/ ./
COPY --from=build /x/application/ ./
USER 10001
EXPOSE 8080
ENTRYPOINT ["java","-XX:MaxRAMPercentage=75", "jar","application.jar"]

Order: least → most frequently changed.
--target build stops early.
```

DOCKER RUN OPTION GROUPS

Group	Options
Process	-d -it --rm --name --entrypoint -w -u --init --restart unless-stopped --stop-timeout
Env	-e K=V --env-file .env
Filesystem	-v name:/p --mount type=... --read-only --tmpfs /tmp
Network	--network n --network-alias a -p 127.0.0.1:8080:8080 -P --add-host --dns
Resources	--memory 512m --cpus 1.5 --pids-limit 200 --ulimit --gpus
Metadata	--label --platform linux/amd64 --log-driver --health-cmd

LIFECYCLE & INSPECTION

```
docker create / start / stop / kill / restart
docker pause / unpause / rm -f / wait
docker ps -a --format 'table {{.Names}}\t{{.Status}}'
docker logs -f --since 10m c
docker exec -it c sh
docker debug c
docker inspect -f '{{.State.ExitCode}}' c
docker top c
docker stats --no-stream
docker cp c:/path .
docker diff c
docker update --memory 1g c
docker events --filter container=c

stop = SIGTERM, wait 10 s, SIGKILL. Exit
137 = killed (OOM?), 143 = SIGTERM,
125-127 = could not start.
```

STORAGE

Kind	Syntax
Named volume	--mount type=volume,src=data,dst=/var/lib/postgresql
Bind mount	--mount type=bind,src="\$(pwd)",dst=/app,readonly (:z/:SELinux)
tmpfs	--mount type=tmpfs,dst=/tmp,tmpfs-size=64m
Image mount	--mount type=image,source=busybox:musl,destination=/dbg
docker volume create[ls]inspect[rm]prune docker run --rm -v data:/d:ro -v "\$PWD"/b \ ubuntu:24.04 tar czf /b/data.tgz -C /d .	

NETWORKING

```
docker network create backend
docker run -d --network backend -name db postgres:18.1
docker run --rm --network backend busybox:1.37 nslookup db
docker network connect frontend api
docker run -p 127.0.0.1:8080:80 nginx:1.29
docker run --add-host=host.docker.internal:host-gateway ...

• User-defined bridge = DNS by name (127.0.0.1); default bridge has none.
• Drivers: bridge, host, none, overlay, macvlan, ipvlan.
• Published ports bypass ufw/firewall; bind to 127.0.0.1 or use DOCKER-USER.
• -link is legacy (no env vars since 29).
```

COMPOSE

```
# compose.yaml (no version: key)
services:
  api:
    build: .
    ports:
      - ["127.0.0.1:8080:8080"]
    environment:
      DB_URL: jdbc:postgresql://db:5432/app
    depends_on:
      db: {condition: service_healthy}
    db:
      image: postgres:18.1
      volumes: [db-
data:/var/lib/postgresql]
      healthcheck:
        test: ["CMD-SHELL","pg_isready -U app"]
      grafana:
        image: grafana/grafana:12.2.0
        profiles: [observability]
      volumes: {db-data: {}}

docker compose up -d --build --wait
docker compose ps | logs -f api |
exec db sh
docker compose run --rm api ./mvnw test
docker compose config #
resolved model
docker compose watch #
develop.watch
docker compose --profile observability up -d
docker compose down -v

compose.override.yaml merges
automatically; include, extends,
${VAR:-default}, secrets in
/run/secrets.
```

BUILDX ESSENTIALS

```
docker buildx create --name ci --driver docker-container --use
docker buildx build --platform linux/amd64,linux/arm64 \
-t reg/app:1.4.0 --push .
docker buildx build \
--cache-from type=gha --cache-to type=gha,mode=max .
docker buildx build --sbom=true --provenance=mode=max --push .
docker build --check .
docker build --secret id=s,src=settings.xml .
docker buildx bake --push #
docker-bake.hcl
docker buildx du / docker builder prune

Caches: inline, registry, local, gha, s3.
Drivers: docker, docker-container,
kubernetes, remote, cloud.
```

SECURITY CHECKLIST

- Non-root USER; never --privileged (use --device).
- --cap-drop ALL (+ only what is needed); keep default seccomp & AppArmor/SELinux.
- --read-only --tmpfs /tmp, --security-opt no-new-privileges.
- Limits: --memory --cpus --pids-limit.
- Secrets as files (--mount=type=secret, Compose secrets), never ENV/ARG.
- Pinned tags + digests; minimal/hardened bases (DHI); docker scout cves.
- SBOM + provenance; sign with Sigstore/Notation (Content Trust retired).
- Protect the socket: SSH contexts or mTLS; rootless mode or users-remap.

CLEAN-UP

```
docker system df -v
docker container prune
docker image prune -a --filter until=168h
docker volume prune #
anonymous; --all for named
docker network prune
docker builder prune --all --filter until=72h
docker system prune --filter label=env=dev
```

SPRING BOOT MAPPING

Need	Use
Image without Dockerfile	./mvnw spring-boot:build-image -Dspring-boot.build-image.imageName=reg/a pp:1.4.0 (Paketo)
Publish from CI	-Dspring-boot.build-image.publish=true -Ddocker.publishRegistry.username=...
Layered Dockerfile	java -Djarmode=tools -jar app.jar extract --layers
Local stack	spring-boot-docker-compose (optional) + compose.yaml
Compose props	spring.docker.compose.file / lifecycle-management / skip.in-tests (true) / profiles.active
Test containers	spring-boot-testcontainers + @ServiceConnection on @Container or @Bean
Other properties	@DynamicPropertySource / DynamicPropertyRegistrar
Dev-time containers	SpringApplication.from(App::main).with(Cfg.class), spring-boot:test-run

TESTCONTAINERS 2.0

```
• Artifacts
org.testcontainers:testcontainers-
module>, testcontainers-
junit-jupiter; classes
org.testcontainers.<module>.*;
no JUnit4.

• Finds Docker via DOCKER_HOST,
tc.host, known sockets;
TESTCONTAINERS_HOST_OVERRIDE;
Ryuk cleans up.

• Waits: forListeningPort / forHttp
/ forLogMessage /
forHealthcheck.

static final CompositeContainer ENV
= new CompositeContainer(

DockerImageName.parse("docker:29.8.1-cli"), composeFile)
.withExposedService("postgres",
5432, Wait.forHealthcheck());
static { ENV.start(); } //
singleton per JVM
//
ENV.getHost("postgres",5432),
getServicePort(...)
```

Maven	Runs
Surefire *Test	mvn test — no Docker
Failsafe *IT	mvn verify — containers; -DskipITs

GITHUB ACTIONS SKELETON

```
permissions: { contents: read,
packages: write }
steps:
  - uses: actions/checkout@v7
  - uses: docker/setup-qemu-action@v4
  - uses: docker/setup-buildx-action@v4
  - uses: docker/login-action@v4
    with: { registry: ghcr.io,
username: ${{ github.actor }},
password: ${{ secrets.GITHUB_TOKEN }} }
  - id: meta
    uses: docker/metadata-action@v6
    with:
      images: ghcr.io/${{ github.repository }}
      tags: |
        type=ref,event=branch
        type=semver,pattern={{version}}
        type=sha
  - uses: docker/build-push-action@v7
    with:
      context: .
      platforms: linux/amd64,linux/arm64
      push: ${{ github.event_name != 'pull_request' }}
      tags: ${{ steps.meta.outputs.tags }}
      labels: ${{ steps.meta.outputs.labels }}
      cache-from: type=gha
      cache-to: type=gha,mode=max

Tests: actions/setup-java@v6 (temurin,
25, cache: maven) + mvn -B -ntp
clean verify on ubuntu-latest;
upload failsafe-reports on failure.
```